

Plush Toy ERP 自动化测试方案

当前状态：产品化内测候选

下一目标：可控客户试用候选

目标：建立一套可复现、可扩展、可进入 CI 的自动化测试体系，覆盖安全、后端领域逻辑、数据库迁移、JSON-RPC API、前端页面、端到端业务流程、客户导入、备份恢复和 release package。

使用方式：可直接用于拆 Codex 任务、补测试目录、配置 CI、制定客户试用前验收标准。

1. 自动化测试总体目标

本项目的自动化测试不是为了追求形式上的覆盖率，而是为了保证 ERP 核心事实不会错。

自动化测试必须优先保护这些风险点：

1. 不能泄漏 secret / token。
2. 生产危险配置必须启动失败。
3. SQL tracing / logging 不能记录 SQL 参数。
4. 运行时不能再出现 `phase8`。
5. `business_records` 不能重新成为正式事实写入源。
6. Workflow 不能直接写库存、出货、采购、财务事实。
7. 库存不能负数。
8. 库存预留不能超预留。
9. 出货不能重复扣库存。
10. 取消出货不能重复回滚库存。
11. 采购收货不能超收。
12. BOM 激活规则必须正确。
13. 销售订单到出货到应收的主链路必须可跑通。
14. 客户导入 dry-run 必须可复现。
15. release package 不能包含敏感信息。
16. 权限控制必须由后端强制执行，不只依赖前端隐藏菜单。

2. 测试分层模型

建议采用如下测试金字塔：

```
      E2E / Playwright / Release Smoke
      -----
    API Contract / JSON-RPC Integration
    -----
  Domain Integration / DB Transaction Tests
  -----
Unit Tests / Pure Functions / Validators / Utils
```

Static Checks / Type / Lint / Secret Scan / Migration Check

每层目标不同：

测试层	目标	是否进 CI	优先级
Static Checks	发现低成本问题	是	P0
Secret Scan	防止敏感信息入库	是	P0
Config Preflight Tests	防止生产危险配置	是	P0
Unit Tests	验证纯逻辑	是	P0
Domain Integration Tests	验证领域事实和事务	是	P0/P1
DB Migration Tests	验证 schema 可迁移	是	P0
JSON-RPC API Tests	验证接口行为和权限	是	P1
Frontend Component Tests	验证页面组件行为	是	P1/P2
E2E Tests	验证核心业务链路	是，分级	P1/P2
Import Tests	验证客户导入 dry-run/apply	是	P0/P1
Security Tests	验证认证、权限、安全头	是	P0/P1
Backup/Restore Tests	验证私有化交付	Nightly / Release	P3
Performance Smoke	验证基础性能退化	Nightly / Release	P3

3. 推荐测试技术栈

3.1 后端 Go

建议使用：

```
testing
httptest
testify/require
testify/assert
testcontainers-go 或 docker compose postgres
go test -race
```

用途：

Usecase 单元测试
Repository 集成测试

JSON-RPC handler 测试
数据库事务测试
并发测试
配置 preflight 测试
权限测试
审计测试

3.2 数据库

建议测试数据库使用：

PostgreSQL container
Atlas migration apply
每个测试独立 schema 或事务回滚

不建议核心库存、出货、采购测试长期使用 SQLite，因为 PostgreSQL 的事务、锁、唯一约束、并发行为才是生产真实行为。

3.3 前端

建议使用：

Vitest
React Testing Library
jsdom
Playwright

用途：

组件渲染
表单校验
列表筛选
权限菜单显示
页面跳转
核心 E2E

3.4 脚本测试

项目已有 Node 脚本测试，继续使用：

```
node --test scripts/**/*.test.mjs
```

用途：

```
project scan
import dry-run
release package
style regression
source manifest
secret scan wrapper
backup report parser
```

3.5 安全扫描

建议使用：

```
gitleaks
trufflehog
grep fallback
自定义 denylist
```

CI 中规则：

```
gitleaks/trufflehog 不存在时，CI 必须失败，不允许 skip。
本地 dev 可以 warn，但 release/CI 必须 fail。
```

4. 测试目录结构建议

建议统一目录：

```
server/
  internal/
    biz/
      *_test.go
    data/
      *_test.go
    service/
      jsonrpc/
        *_test.go
    conf/
      preflight_test.go
```

```
test/
  integration/
    inventory_test.go
    shipment_test.go
    purchase_test.go
    sales_test.go
    bom_test.go
    finance_test.go
    workflow_test.go
  e2e/
    manufacturing_mvp_test.go
testkit/
  db.go
  fixtures.go
  builders.go
  jsonrpc_client.go
  assertions.go

web/
  src/
    __tests__/
  erp/
    pages/
      *.test.jsx
    components/
      *.test.jsx
  e2e/
    auth.spec.ts
    sales-order.spec.ts
    inventory.spec.ts
    shipment.spec.ts
    mvp-flow.spec.ts

scripts/
  import/
    *.test.mjs
  qa/
    *.test.mjs
  release/
    *.test.mjs
  security/
    *.test.mjs
  e2e/
    manufacturing-mvp.mjs
```

5. 测试命名规范

5.1 Go 测试命名

建议格式：

```
func Test<Domain>_<Action>_<ExpectedResult>(t *testing.T)
```

示例：

```
func TestInventoryReservation_Create_RejectsOverReservation(t *testing.T)
func TestShipment_Ship_DoesNotDoubleDeductOnRetry(t *testing.T)
func TestPurchaseReceipt_Create_RejectsOverReceive(t *testing.T)
func TestBOM_Activate_RejectsSecondActiveVersion(t *testing.T)
```

5.2 前端测试命名

建议格式：

```
<PageName>.<behavior>.test.jsx
```

示例：

```
SalesOrderPage.create-draft.test.jsx
InventoryBalancePage.filter.test.jsx
ShipmentPage.ship-action.test.jsx
RolePermission.menu-visibility.test.jsx
```

5.3 E2E 测试命名

建议格式：

```
<business-flow>.spec.ts
```

示例：

```
manufacturing-mvp.spec.ts
purchase-to-stock.spec.ts
```

```
sales-to-shipment.spec.ts
shipment-to-receivable.spec.ts
```

6. 测试数据策略

6.1 不要在测试里散落手写数据

建议建立统一 test builder：

```
server/test/testkit/builders.go
```

包含：

```
NewCustomerBuilder()
NewSupplierBuilder()
NewProductBuilder()
NewSKUBuilder()
NewMaterialBuilder()
NewBOMBuilder()
NewSalesOrderBuilder()
NewPurchaseOrderBuilder()
NewInventoryBalanceBuilder()
NewShipmentBuilder()
```

6.2 测试数据必须可读

示例：

```
customer: "Test Customer A"
supplier: "Test Supplier A"
product: "Plush Bear"
sku: "PLUSH-BEAR-PINK-S"
material: "Pink Fabric"
warehouse: "Main Warehouse"
```

不要使用无意义数据：

```
abc
test1
```

```
foo
bar
```

6.3 所有测试必须确定性

禁止：

```
依赖当前时间导致结果不稳定
依赖测试运行顺序
依赖外部网络
依赖真实 token
依赖真实客户原始文件
依赖本机路径
```

建议：

```
统一 fake clock
统一 test IDs
统一 fixture path
统一 timezone
统一 random seed
```

6.4 时间测试要求

ERP 状态很依赖日期，测试中必须显式设定：

```
order_date
expected_arrival_date
delivery_date
shipment_date
created_at
updated_at
```

不要依赖 `time.Now()` 直接决定断言。

7. P0 自动化测试任务

P0-T01：Secret Scan 自动化测试

目标

保证源码和 release package 中不包含真实 secret。

需要覆盖

```
npm token
yarn token
Telegram bot token
JWT secret
DB password
private key
.env
webhook
access token
refresh token
API key
```

建议测试文件

```
scripts/security/secret-scan.test.mjs
scripts/qa/secrets.sh
```

测试场景

场景	预期
存在 <code>_authToken=real-token</code>	fail
存在 <code>BEGIN PRIVATE KEY</code>	fail
存在 <code>bot123456:ABC</code>	fail
<code>.npmrc.example</code> 使用 <code>\${NPM_TOKEN}</code>	pass
<code>.env.example</code> 使用 placeholder	pass
release package 含 <code>.env</code>	fail
gitleaks 不存在且 CI=true	fail

CI 命令

```
bash scripts/qa/secrets.sh
node --test scripts/security/*.test.mjs
```

Codex 提示词

请实现 P0-T01 : Secret Scan 自动化测试。

要求：

1. 新增 scripts/security/secret-scan.test.mjs。
2. 覆盖 npm token、private key、Telegram bot token、真实 .env、JWT secret、DB password。
3. 修改 scripts/qa/secrets.sh, 使 CI=true 时 gitleaks/trufflehog 缺失必须失败。
4. example 文件中的 \${NPM_TOKEN}、change-this placeholder 不应误报, 但 release/prod 真实文件不能出现。
5. 运行 node --test scripts/security/*.test.mjs 和 bash scripts/qa/secrets.sh。

P0-T02 : Production Preflight 测试

目标

生产、预发、客户试用环境下，危险配置必须启动失败。

建议测试文件

server/internal/conf/preflight_test.go

测试场景

场景	环境	预期
JWT secret 为空	production	fail
JWT secret 是 change-this	production	fail
DB password 是 postgres	production	fail
Admin password 是 12345678	production	fail
SMS mock enabled	production	fail
debug seed enabled	production	fail
debug cleanup enabled	production	fail
public register enabled	production	fail
CORS *	production	fail
SQL args tracing enabled	production	fail
backup path empty	private-deploy	fail
dev 环境使用 mock SMS	dev	pass 或 warn

场景	环境	预期
test 环境使用 placeholder	test	pass

Go 测试命令

```
cd server
go test ./internal/conf -run TestPreflight
```

Codex 提示词

请实现 P0-T02 : Production Preflight 测试。

要求：

1. 在 server/internal/conf/preflight_test.go 中覆盖 production/staging/customer-trial/private-deploy。
2. 每个危险配置都必须有独立测试。
3. dev/test 环境可允许安全降级，但 production 必须失败。
4. 错误信息必须能说明失败原因。
5. 运行 go test ./internal/conf -run TestPreflight。

P0-T03 : SQL Tracing Redaction 测试

目标

确保 SQL 参数不会进入 trace/log。

建议测试文件

```
server/internal/data/tracing_test.go
server/internal/conf/preflight_test.go
```

测试场景

场景	预期
production 配置开启 SQL args tracing	preflight fail
tracing attribute 中出现 db.sql.arg.0	fail
tracing attribute 中出现手机号/地址/password	fail
tracing 只记录 statement hash	pass
dev 默认不记录 args	pass
local debug 显式开启时敏感字段被 redact	pass

断言

禁止出现：

```
db.sql.arg.*  
password  
token  
secret  
phone  
address  
raw_import_value
```

允许出现：

```
db.system  
db.operation  
db.statement.hash  
db.duration_ms  
db.rows_affected
```

Codex 提示词

请实现 P0-T03：SQL Tracing Redaction 测试。

要求：

1. 找到数据库 tracing/Ent debug 相关代码。
2. 增加测试，验证 trace attributes 不包含 db.sql.arg.*。
3. 增加测试，验证 production 开启 SQL args tracing 会被 preflight 拒绝。
4. 如果保留本地 debug 模式，测试 password/token/phone/address 会被 redact。
5. 运行相关 go test。

P0-T04：客户导入 Manifest 测试

目标

导入 dry-run 不依赖中文文件名，必须通过 source manifest 稳定定位文件。

建议测试文件

```
scripts/import/source-manifest.test.mjs  
scripts/import/yoyoosun-import.test.mjs
```

测试场景

场景	预期
manifest 存在 required source	pass
required source 缺失	fail, 错误明确
sha256 不匹配	fail
originalName 中文但 storedName 英文	pass
文件名变成 #Uxxxx	不影响, 通过 storedName 定位
sourceId 重复	fail
type 缺失	fail
storedName 缺失	fail
dry-run sourceCount > 0	pass

Node 测试命令

```
node --test scripts/import/*.test.mjs
node --test scripts/**/*.test.mjs
```

Codex 提示词

请实现 P0-T04 : 客户导入 Manifest 测试。

要求：

1. 新增 source-manifest.test.mjs。
2. 覆盖 required source、sha256、sourceId 唯一、storedName、type 校验。
3. 修改导入 dry-run, 使其优先通过 manifest 的 storedName/sourceId 定位文件。
4. 测试 originalName 为中文时不影响导入。
5. 运行 node --test scripts/import/*.test.mjs。

P0-T05 : CI Baseline 测试命令

目标

让所有基础测试能在 CI 中一键跑起来。

最小 CI 命令

```
bash scripts/project-scan.sh --strict
bash scripts/qa/secrets.sh
node --test scripts/**/*.test.mjs
```

```
cd web
pnpm install --frozen-lockfile
pnpm lint:check
pnpm test
pnpm build

cd ../server
go test ./...
```

推荐新增脚本

```
scripts/qa/ci.sh
scripts/qa/backend.sh
scripts/qa/frontend.sh
scripts/qa/scripts.sh
scripts/qa/security.sh
```

验收标准

- 本地可以执行 `bash scripts/qa/ci.sh`。
- CI 使用同一套脚本。
- CI 不直接复制大量命令，避免本地和 CI 漂移。
- 任何一步失败都整体失败。

Codex 提示词

请实现 P0-T05 : CI Baseline 测试脚本。

要求：

1. 新增 `scripts/qa/ci.sh`、`backend.sh`、`frontend.sh`、`scripts.sh`、`security.sh`。
2. CI workflow 调用这些脚本，而不是复制大量命令。
3. 每个脚本 `set -euo pipefail`。
4. 任意测试失败必须退出非 0。
5. 本地和 CI 使用同一套入口。

8. P1 自动化测试任务

P1-T01：禁止 Runtime Phase8 测试

目标

确保产品运行时代码、API、前端路由不再出现 `phase8`。

建议测试文件

```
scripts/qa/no-runtime-phase8.test.mjs
```

允许出现的路径

```
docs/archive/  
docs/evidence/  
docs/history/
```

禁止出现的路径

```
server/internal/  
web/src/  
web/scripts/  
config/  
deploy/
```

测试场景

场景	预期
<code>server/internal/data/jsonrpc_phase8.go</code> 存在	fail
<code>phase8.createShipment</code> 出现在 API 测试	fail
<code>/erp/phase8/facts</code> 出现在前端路由	fail
<code>docs/archive/phase8.md</code>	pass

Codex 提示词

请实现 P1-T01：禁止 runtime phase8 测试。

要求：

1. 新增 `scripts/qa/no-runtime-phase8.test.mjs`。
2. 扫描 `server/internal`、`web/src`、`config`、`deploy`，禁止 `phase8/Phase8/PHASE8`。
3. `docs/archive`、`docs/evidence` 可以例外。
4. 把该测试加入 `node --test scripts/**/*.test.mjs`。

P1-T02：Business Records 只读测试

目标

确保 `business_records` 不再作为正式业务写入源。

建议测试文件

```
server/internal/biz/business_records_test.go
server/internal/service/jsonrpc/business_records_test.go
scripts/qa/no-business-records-write.test.mjs
```

测试场景

场景	预期
调用 CreateRecord	fail 或返回 archive read-only
调用 UpdateRecord	fail
调用 DeleteRecord	fail
正式销售订单创建不写 business_records	pass
正式库存流水不写 business_records	pass
dashboard 不从 business_records 读正式 KPI	pass
Legacy Archive 只读查询	pass

Codex 提示词

请实现 P1-T02 : business_records 只读测试。

要求：

1. 增加后端测试，验证 Create/Update/Delete business_records 都被拒绝。
2. 增加测试，验证正式销售、库存、出货、财务 usecase 不写 business_records。
3. 增加脚本扫描，禁止新增 business_records write path。
4. Legacy Archive 只读查询允许。

P1-T03：Workflow 不直接写业务事实测试

目标

确保 workflow 只做任务流，不直接写库存、出货、采购、财务事实。

建议测试文件

```
scripts/qa/workflow-boundary.test.mjs
server/internal/biz/workflow_boundary_test.go
```

禁止依赖

Workflow 不应该直接依赖或调用：

```
InventoryTxnRepo.Create  
ShipmentRepo.Ship  
FinanceFactRepo.Create  
PurchaseReceiptRepo.Create  
ProductionFactRepo.Create  
OutsourcingFactRepo.Create
```

允许行为

```
创建任务  
完成任务  
评论  
转交  
引用业务对象 ID  
调用领域 usecase 的 command gateway, 前提是边界清晰
```

测试场景

场景	预期
workflow task complete 直接创建 inventory_txn	fail
workflow task complete 直接创建 finance_fact	fail
workflow task 只更新 workflow_tasks	pass
workflow task 触发明确 domain command	pass, 但需测试 domain usecase

Codex 提示词

请实现 P1-T03 : Workflow 边界测试。

要求：

1. 增加脚本扫描 workflow 包, 禁止直接写 inventory_txns、shipments、finance_facts、purchase_receipts。
2. 增加后端测试, 验证完成 workflow task 不会直接产生库存/财务/出货事实。
3. 如果存在通过 domain command 触发领域 usecase, 必须有明确接口和测试。

P1-T04：库存预留并发测试

目标

并发预留不能超过可用库存。

建议测试文件

```
server/test/integration/inventory_reservation_concurrency_test.go
```

必须使用 PostgreSQL

该测试必须使用 PostgreSQL，因为 SQLite 无法可靠模拟生产锁行为。

测试数据

```
SKU: PLUSH-BEAR-PINK-S
Warehouse: MAIN
Inventory available: 100
Concurrent requests: 10
Each request reserve: 20
Expected successful reservations: 5
Expected total reserved: 100
Expected failed reservations: 5
```

测试场景

场景	预期
10 个请求并发各预留 20，总库存 100	只有 5 个成功
2 个请求并发各预留 80，总库存 100	只有 1 个成功
使用同一 idempotency key 重试	返回同一结果，不新增
取消预留后重新预留	pass
取消两次	no-op 或明确 fail，不释放两次

Go 测试命令

```
cd server
go test ./test/integration -run TestInventoryReservationConcurrency -count=20
```

Codex 提示词

```
请实现 P1-T04：库存预留并发测试。
```

要求：

1. 使用 PostgreSQL 测试库。
2. 构造库存 available=100。
3. 并发 10 个 goroutine, 每个预留 20。
4. 断言最多 5 个成功, active reserved 总数不超过 100。
5. 增加重复 idempotency key 测试。
6. 增加取消预留测试。
7. 该测试至少 -count=20 稳定通过。

P1-T05：库存负数保护测试

目标

任何出库、发料、出货、退货都不能导致库存余额为负。

建议测试文件

```
server/test/integration/inventory_negative_guard_test.go
```

测试场景

业务动作	库存	请求数量	预期
出货	10	20	fail
生产领料	10	20	fail
外协发料	10	20	fail
采购退货	10	20	fail
库存调整出库	10	20	fail, 除非特殊权限
取消已发货回滚	0	+10	pass

验收标准

- 失败时库存余额不变。
- 失败时不生成库存流水。
- 成功时余额和流水一致。

Codex 提示词

请实现 P1-T05：库存负数保护测试。

要求：

1. 覆盖 shipment、production consume、outsourcing issue、purchase return、stock adjustment。

2. 出库数量超过可用库存时必须失败。
3. 失败时库存余额不变，库存流水不新增。
4. 成功动作要断言余额和流水一致。

P1-T06：出货幂等测试

目标

重复请求不能造成重复扣库存或重复回滚。

建议测试文件

```
server/test/integration/shipment_idempotency_test.go
```

测试场景

场景	预期
重复 add item 同一个 sales_order_item	只生成一行或返回已有行
重复 ship 同一个出货单	只扣一次库存
重复 cancel shipped	只回滚一次库存
ship 后再 add item	fail
空出货单 ship	fail
未发货出货单 cancel shipped	fail
已取消出货单再次 ship	fail

关键断言

```
shipment_items 行数
inventory_txns 行数
inventory_balance 数量
shipment.status
sales_order_item.shipped_qty
finance_facts 行数
```

Codex 提示词

请实现 P1-T06：出货幂等测试。

要求：

1. 覆盖 add item、ship、cancel shipped 的重复请求。

2. 断言库存流水不会重复生成。
3. 断言库存余额不会重复扣减或重复回滚。
4. 断言销售订单已发数量不会重复增加。
5. 断言应收事实不会重复生成。

P1-T07：采购订单和收货数量测试

目标

采购收货必须引用采购订单明细，并且不能超收。

建议测试文件

```
server/test/integration/purchase_order_receipt_test.go
```

测试场景

场景	预期
DRAFT PO 创建明细	pass
DRAFT PO 收货	fail
APPROVED PO 收货 40/100	PARTIALLY_RECEIVED
再收货 60/100	RECEIVED
再收货 1	fail
收货数量为 0	fail
取消 PO 后收货	fail
CLOSED PO 后收货	fail

断言

```
purchase_order.status  
purchase_order_item.received_qty  
purchase_receipts rows  
inventory_txns rows  
quality_inspections rows
```

Codex 提示词

请实现 P1-T07：采购订单和收货数量测试。

要求：

1. 测试 P0 状态机。
2. 测试收货必须引用 purchase_order_item。
3. 测试部分收货、完全收货、超收失败。
4. 测试取消/关闭后不能收货。
5. 验证库存和质检的后续影响。

P1-T08：BOM 状态机测试

目标

BOM 版本规则必须稳定。

建议测试文件

```
server/test/integration/bom_version_test.go
```

测试场景

场景	预期
创建 DRAFT BOM	pass
DRAFT 添加 item	pass
item qty <= 0	fail
loss_rate < 0	fail
激活 BOM	ACTIVE
同 SKU 第二个 ACTIVE BOM	fail 或旧版本自动归档，按产品规则
ACTIVE BOM 修改 item	fail
copy new version	DRAFT
ARCHIVED BOM 修改	fail
销售订单生成采购需求只使用 ACTIVE BOM	pass

Codex 提示词

请实现 P1-T08：BOM 状态机测试。

要求：

1. 覆盖 DRAFT/ACTIVE/ARCHIVED。
2. 覆盖同一个 product/sku 只能有一个 ACTIVE BOM。
3. 覆盖 ACTIVE 不允许直接修改 item。

4. 覆盖 copy new version。
5. 覆盖采购需求展开只使用 ACTIVE BOM。

P1-T09：权限后端强制测试

目标

权限必须由后端强制执行，不只依赖前端菜单隐藏。

建议测试文件

```
server/internal/service/jsonrpc/rbac_test.go
```

角色

```
super_admin
admin
sales
purchase
warehouse
quality
production
outsourcing
finance
viewer
```

测试场景

角色	API	预期
viewer	sales.createOrder	fail
sales	sales.createOrder	pass
sales	purchase.approveOrder	fail
purchase	purchase.approveOrder	pass
warehouse	shipment.ship	pass
finance	finance.createFact	pass
viewer	finance.createFact	fail
quality	quality.submitInspection	pass
production	production.createReport	pass
anonymous	protected API	fail

角色	API	预期
disabled user	any API	fail

Codex 提示词

请实现 P1-T09：权限后端强制测试。

要求：

1. 构造不同角色用户。
2. 通过 JSON-RPC 调用核心 API。
3. 验证没有权限的角色返回 forbidden。
4. 验证前端菜单隐藏不是唯一权限控制。
5. 覆盖 disabled user 和 anonymous。

P1-T10：审计日志测试

目标

关键业务动作必须写 audit event。

建议测试文件

```
server/test/integration/audit_event_test.go
```

必须审计的动作

```
login
create customer
update customer
create sales order
confirm sales order
approve purchase order
receive goods
quality pass/reject
inventory adjustment
create reservation
cancel reservation
ship shipment
cancel shipped shipment
create finance fact
bootstrap admin
change role permission
run import
```

测试断言

```
audit_events.action
audit_events.actor_id
audit_events.object_type
audit_events.object_id
audit_events.before
audit_events.after
audit_events.created_at
```

Codex 提示词

请实现 P1-T10：审计日志测试。

要求：

1. 覆盖关键业务动作。
2. 每个动作执行后断言 `audit_events` 中有记录。
3. `audit event` 必须包含 `actor`、`action`、`object_type`、`object_id`、`created_at`。
4. 对 `update` 类动作，尽量包含 `before/after`。

9. P2 自动化测试任务

P2-T01：ERP MVP 端到端后端测试

目标

用后端 integration test 跑通制造业主链路。

建议测试文件

```
server/test/e2e/manufacturing_mvp_test.go
```

流程

1. 创建客户
2. 创建供应商
3. 创建产品
4. 创建 SKU
5. 创建物料
6. 创建 BOM DRAFT
7. 激活 BOM
8. 创建销售订单

9. 确认销售订单
10. 根据 BOM 生成采购需求
11. 创建采购订单
12. 审核采购订单
13. 创建采购收货
14. 创建质检记录 PASS
15. 入库
16. 创建生产任务
17. 生产领料
18. 生产上报
19. 成品入库
20. 创建库存预留
21. 创建出货单
22. 发货
23. 生成应收事实
24. 登记收款
25. 关闭订单

必须断言

```
sales_order.status  
purchase_order.status  
inventory_balance  
inventory_txns count  
production_task.status  
shipment.status  
finance_fact.status  
audit_events count  
business_records not written  
phase8 API not used
```

输出报告

```
{  
  "customerCreated": true,  
  "skuCreated": true,  
  "bomActivated": true,  
  "salesOrderConfirmed": true,  
  "purchaseOrderReceived": true,  
  "qualityPassed": true,  
  "inventoryIncreased": true,  
  "productionCompleted": true,  
  "reservationCreated": true,  
  "shipmentShipped": true,  
  "receivableCreated": true,  
  "businessRecordsWriteCount": 0,  
  "phase8Used": false  
}
```

Codex 提示词

请实现 P2-T01 : ERP MVP 端到端后端测试。

要求：

1. 在 `server/test/e2e/manufacturing_mvp_test.go` 中实现完整链路。
2. 不允许使用 `business_records` 写入。
3. 不允许使用 `phase8` API。
4. 对每一步状态和事实源做断言。
5. 输出 JSON report 或测试日志。
6. 加入 `qa full` 或 `CI extended job`。

P2-T02 : JSON-RPC Contract 测试

目标

确保 JSON-RPC API 格式、错误码、权限和参数校验稳定。

建议测试文件

```
server/internal/service/jsonrpc/contract_test.go
```

需要统一的响应结构

```
{
  "jsonrpc": "2.0",
  "id": "request-id",
  "result": {}
}
```

错误结构：

```
{
  "jsonrpc": "2.0",
  "id": "request-id",
  "error": {
    "code": "FORBIDDEN",
    "message": "permission denied",
    "details": {}
  }
}
```

测试场景

场景	预期
method 缺失	INVALID_REQUEST
params 类型错误	INVALID_PARAMS
未登录访问 protected method	UNAUTHORIZED
无权限访问	FORBIDDEN
业务校验失败	BUSINESS_ERROR
重复 idempotency key	返回已有结果
未知 method	METHOD_NOT_FOUND
public method	无 token 可访问
protected method	需要 token

Codex 提示词

请实现 P2-T02 : JSON-RPC Contract 测试。

要求：

- 1. 覆盖 request/response 格式。
- 2. 覆盖 invalid request、invalid params、unauthorized、forbidden、business error。
- 3. 覆盖 unknown method。
- 4. 覆盖 public/protected method。
- 5. 覆盖 idempotency key。
- 6. 不依赖 phase8 method。

P2-T03：前端核心页面组件测试

目标

核心页面的基础交互不能回归。

测试范围

客户管理
供应商管理
产品管理
SKU 管理
BOM 管理
销售订单
采购订单

采购收货
质检任务
库存余额
库存预留
出货单
应收/应付事实
用户管理
角色权限
审计日志

每个页面至少测试

测试	说明
页面能渲染	不崩溃
列表加载态	loading
空数据态	empty
错误态	error
筛选	filter
创建表单打开	create modal/drawer
必填校验	required validation
权限隐藏按钮	RBAC UI
点击详情	detail drawer/page
API 错误提示	message/notification

示例文件

```
web/src/erp/pages/sku/SkuPage.test.jsx  
web/src/erp/pages/sales/SalesOrderPage.test.jsx  
web/src/erp/pages/shipment/ShipmentPage.test.jsx
```

Codex 提示词

请实现 P2-T03：前端核心页面组件测试。

要求：

1. 使用 Vitest 和 React Testing Library。
2. 先选择 SKU 管理、销售订单、出货单三个页面做样板。
3. 覆盖渲染、loading、empty、error、filter、create form、required validation、permission button。
4. mock JSON-RPC client，不连接真实后端。
5. 运行 pnpm test。

P2-T04：Playwright 核心 E2E 测试

目标

从浏览器角度验证用户可以完成核心流程。

建议文件

```
web/e2e/manufacturing-mvp.spec.ts
web/e2e/auth.spec.ts
web/e2e/rbac.spec.ts
web/e2e/shipment.spec.ts
```

MVP 浏览器流程

1. 登录
2. 创建客户
3. 创建产品
4. 创建 SKU
5. 创建 BOM
6. 创建销售订单
7. 确认销售订单
8. 创建采购订单
9. 收货
10. 质检通过
11. 查看库存增加
12. 创建出货单
13. 发货
14. 查看库存减少
15. 查看应收事实

E2E 分级

等级	频率	内容
smoke	每次 PR	登录、首页、一个只读页面
core	main 分支	销售到出货主流程
full	nightly/release	完整制造链路
release	发布前	客户试用包 smoke

Codex 提示词

请实现 P2-T04：Playwright 核心 E2E 测试。

要求：

1. 新增 web/e2e/auth.spec.ts 和 web/e2e/manufacturing-mvp.spec.ts。
2. 先实现 smoke：登录、进入首页、打开销售订单列表。
3. 再实现 core flow：创建销售订单到出货。
4. 使用测试账号和测试 seed，不依赖真实客户数据。
5. 失败时保留 screenshot 和 trace。

P2-T05：前端权限菜单测试

目标

不同角色只能看到自己有权限的菜单和动作。

建议文件

```
web/src/erp/auth/permission-menu.test.jsx
web/e2e/rbac.spec.ts
```

测试角色

```
sales
purchase
warehouse
quality
production
finance
viewer
admin
```

测试场景

角色	应显示	不应显示
sales	销售订单、客户	系统配置、角色权限
purchase	采购订单、供应商	财务配置
warehouse	库存、出货	角色权限
quality	质检任务	应收事实
finance	应收、应付	生产上报
viewer	只读菜单	新建/审核按钮
admin	系统管理	无

注意

该测试只验证前端显示，不替代后端 RBAC 测试。

Codex 提示词

请实现 P2-T05：前端权限菜单测试。

要求：

1. 构造不同角色的权限数据。

2. 测试菜单显示和关键按钮显示。

3. viewer 不应看到新建/审核/删除按钮。

4. admin 应看到系统管理。

5. 明确注释：后端权限由 server RBAC tests 保证。

10. P3 自动化测试任务

P3-T01：Release Package 测试

目标

客户试用包可生成、可扫描、可验收。

建议测试文件

scripts/release/release-package.test.mjs

测试场景

场景	预期
release 目录生成	pass
包含 .env.example	pass
不包含 .env	pass
不包含 .npmrc 真实 token	pass
包含 migration-status.txt	pass
包含 import-dry-run-report.json	pass
包含 smoke-test-report.json	pass
包含 security-scan-report.json	pass

场景	预期
包含 known-limitations.md	pass
包含 acceptance-checklist.md	pass
source.tar.gz 不含 node_modules	pass
source.tar.gz 不含 dist/cache	pass

Codex 提示词

请实现 P3-T01 : Release Package 测试。

要求：

1. 新增 scripts/release/release-package.test.mjs。
2. 测试 release package 文件结构。
3. 测试不能包含 .env、真实 .npmrc、token、node_modules、dist/cache。
4. 测试必须包含 known-limitations、acceptance-checklist、migration-status、import dry-run report。
5. 将测试加入 release build 前置。

P3-T02：备份恢复自动化测试

目标

验证私有化部署可以备份、恢复和 smoke test。

建议文件

```
scripts/deploy/backup-restore.test.mjs
scripts/deploy/verify-restore.sh
```

测试流程

1. 启动测试 PostgreSQL
2. 执行 migration
3. 写入测试数据
4. 执行 backup
5. 删除数据库
6. restore backup
7. 执行 smoke query
8. 验证关键数据存在
9. 输出 backup-restore-report.json

验收标准

```
backup file exists
backup hash exists
restore success
migration version matches
smoke query pass
report generated
```

Codex 提示词

请实现 P3-T02：备份恢复自动化测试。

要求：

1. 使用测试数据库，不要连接生产库。
2. 备份后恢复到新数据库。
3. 验证 migration version 和 smoke 数据。
4. 输出 backup-restore-report.json。
5. 不要把备份文件提交到仓库。

P3-T03：性能 Smoke 测试

目标

不是做完整压测，而是防止明显性能退化。

建议文件

```
scripts/perf/smoke.test.mjs
scripts/perf/jsonrpc-smoke.mjs
```

初始指标

API	数据量	目标
登录	1 user	p95 < 500ms
销售订单列表	1000 orders	p95 < 1000ms
库存余额列表	5000 rows	p95 < 1200ms
出货单详情	100 items	p95 < 1000ms
首页 KPI	mock 10k facts	p95 < 1500ms

注意

这些数字是 MVP smoke target，不是最终 SLA。

如果环境较弱，可以先作为 warn，不作为 hard fail。release 前再变成 hard gate。

Codex 提示词

请实现 P3-T03：性能 Smoke 测试。

要求：

1. 新增 scripts/perf/jsonrpc-smoke.mjs。
2. 支持配置 API endpoint、token、并发数、数据量。
3. 输出 p50/p95/p99。
4. 先对登录、销售订单列表、库存余额列表、出货单详情、首页 KPI 做 smoke。
5. CI PR 可跳过, nightly/release 运行。

11. 菜单页面测试矩阵

11.1 主数据测试矩阵

页面	单元测试	API 测试	前端测试	E2E	必测重点
客户管理	是	是	是	是	编号唯一、禁用后不可下单
供应商管理	是	是	是	是	禁用后不可采购
联系人管理	是	是	是	否	默认联系人
产品管理	是	是	是	是	产品状态、SKU 关联
SKU 管理	是	是	是	是	SKU code 唯一、库存引用
物料管理	是	是	是	是	物料类别、安全库存
单位管理	是	是	是	否	单位换算
仓库库位	是	是	是	是	禁用仓库不可入库
BOM 管理	是	是	是	是	版本、激活、不可修改 ACTIVE

11.2 销售采购测试矩阵

页面	单元测试	API 测试	前端测试	E2E	必测重点
销售订单	是	是	是	是	状态机、明细、确认
销售订单明细	是	是	是	是	已发数量、未发数量

页面	单元测试	API 测试	前端测试	E2E	必测重点
采购需求	是	是	是	是	BOM 展开、库存扣减
采购订单	是	是	是	是	审核、取消、关闭
采购收货	是	是	是	是	不能超收、生成质检/库存
采购退货	是	是	是	是	库存扣减、来源回溯

11.3 库存生产出货测试矩阵

页面	单元测试	API 测试	前端测试	E2E	必测重点
质检任务	是	是	是	是	PASS/REJECT/PARTIAL
库存余额	是	是	是	是	可用库存计算
库存批次	是	是	是	是	批次状态
库存流水	是	是	是	是	不可删除、反向修正
库存预留	是	是	是	是	并发、幂等、释放
库存调整	是	是	是	否	审核后生成流水
生产任务	是	是	是	是	BOM、缺料、完工
生产上报	是	是	是	是	完工数量、入库
外协任务	是	是	是	是	发料、回货、应付
出货单	是	是	是	是	扣库存、幂等、回滚

11.4 财务系统测试矩阵

页面	单元测试	API 测试	前端测试	E2E	必测重点
应收事实	是	是	是	是	出货生成、收款回写
应付事实	是	是	是	是	采购/外协生成、付款回写
收付款记录	是	是	是	是	确认后回写
财务边界	否	否	是	否	UI 不误导成总账

11.5 系统管理测试矩阵

页面	单元测试	API 测试	前端测试	E2E	必测重点
用户管理	是	是	是	是	禁用用户不可登录
角色权限	是	是	是	是	后端强制权限
审计日志	是	是	是	是	关键操作记录
客户配置	是	是	是	否	菜单/品牌配置
系统配置	是	是	是	否	不显示 secret 明文
备份恢复	是	是	是	是	restore 验证

12. 关键领域测试用例清单

12.1 销售订单测试

- [] 创建 DRAFT 销售订单
- [] DRAFT 可编辑
- [] CONFIRMED 后不可随意编辑关键字段
- [] 无明细不能确认
- [] SKU 禁用不能下单
- [] 客户禁用不能下单
- [] 确认订单生成采购需求或生产建议
- [] 部分出货后状态 PARTIALLY_SHIPPED
- [] 全部出货后状态 SHIPPED
- [] 取消订单释放预留
- [] 关闭订单后只读
- [] 订单确认写 audit event

12.2 BOM 测试

- [] 创建 DRAFT BOM
- [] DRAFT 可添加物料
- [] 物料 qty 必须 > 0
- [] loss_rate 不可为负
- [] 激活 BOM 成功
- [] 同 SKU 不能有两个 ACTIVE BOM
- [] ACTIVE BOM 不可直接修改
- [] copy new version 后为 DRAFT
- [] ARCHIVED BOM 只读
- [] 销售订单采购需求只使用 ACTIVE BOM

12.3 采购测试

- ☐ 创建 DRAFT PO
- ☐ APPROVED 后可收货
- ☐ DRAFT 不可收货
- ☐ CANCELLED 不可收货
- ☐ CLOSED 不可收货
- ☐ 部分收货更新 PARTIALLY_RECEIVED
- ☐ 完全收货更新 RECEIVED
- ☐ 超收失败
- ☐ 收货数量为 0 失败
- ☐ 收货后生成质检任务或库存流水
- ☐ 收货写 audit event

12.4 质量检测

- ☐ 采购收货生成质检任务
- ☐ 质检 PASS 允许入库
- ☐ 质检 REJECT 进入拒收/退货
- ☐ PARTIAL_PASS 合格部分入库
- ☐ 合格数量 + 不合格数量 不得超过待检数量
- ☐ 重复提交质检失败或幂等
- ☐ 质检写 audit event

12.5 库存测试

- ☐ 入库增加库存余额
- ☐ 出库减少库存余额
- ☐ 出库不能导致负库存
- ☐ 错误库存通过反向流水修正
- ☐ 库存流水不可删除
- ☐ 库存余额等于流水聚合结果
- ☐ 批次状态影响可用库存
- ☐ 待检库存不计入可用库存
- ☐ 不良库存不计入可用库存
- ☐ 调整必须生成库存流水

12.6 库存预留测试

- ☐ 可用库存足够时创建预留
- ☐ 可用库存不足时创建失败
- ☐ 并发预留不能超过可用库存
- ☐ 重复 idempotency key 不重复创建
- ☐ 取消预留释放可用库存
- ☐ 重复取消不重复释放
- ☐ 出货消耗预留
- ☐ 预留过期后不可被出货使用

12.7 生产测试

- ☐ 销售订单可生成生产任务
- ☐ 生产任务必须引用 ACTIVE BOM
- ☐ 无 BOM 不能下达生产
- ☐ 缺料时提示风险
- ☐ 领料生成库存出库流水
- ☐ 上报数量不能超过计划数量
- ☐ 部分完工更新 PARTIALLY_COMPLETED
- ☐ 完全完工更新 COMPLETED
- ☐ 合格品入库
- ☐ 不良品进入异常

12.8 外协测试

- ☐ 创建外协任务
- ☐ 外协厂禁用不可创建任务
- ☐ 外协发料扣减库存
- ☐ 发料不能导致负库存
- ☐ 外协回货生成质检或入库
- ☐ 部分回货更新 PARTIALLY_RETURNED
- ☐ 全部回货更新 RETURNED
- ☐ 外协回货生成应付事实
- ☐ 取消任务不能丢失已发料事实

12.9 出货测试

- ☐ 从销售订单创建出货单
- ☐ 出货明细不能超过未发数量

- ☐ 出货库存不足失败
- ☐ 出货消耗预留
- ☐ 确认发货扣库存
- ☐ 重复确认发货不重复扣库存
- ☐ 已发货取消生成反向库存流水
- ☐ 重复取消不重复回滚
- ☐ 全部发货更新销售订单 SHIPPED
- ☐ 出货生成应收事实

12.10 财务事实测试

- ☐ 出货生成应收事实
- ☐ 采购收货生成应付事实
- ☐ 外协回货生成应付事实
- ☐ 重复来源不重复生成财务事实
- ☐ 收款记录确认后更新应收已收金额
- ☐ 付款记录确认后更新应付已付金额
- ☐ 全额收款后状态 PAID
- ☐ 全额付款后状态 PAID
- ☐ 取消财务事实必须可追溯

13. CI 分层设计

13.1 PR CI

每个 PR 必须跑：

```
secret scan
project scan
script tests
backend unit tests
backend fast integration tests
frontend lint
frontend unit/component tests
frontend build
migration lint/status
```

目标时间：

5 到 15 分钟内

13.2 Main CI

合并到 main 后跑：

```
PR CI 全部内容
PostgreSQL integration tests
JSON-RPC contract tests
核心 E2E smoke
docker build
release dry-run
```

13.3 Nightly CI

每天跑：

```
完整后端 integration
完整 Playwright E2E
性能 smoke
备份恢复演练
导入 dry-run 全量
release package build
secret scan full tree
go test -race
```

13.4 Release CI

准备客户试用包前跑：

```
P0 全部
P1 全部
P2 MVP e2e
release package test
backup restore
import dry-run
security scan full tree
known limitations check
acceptance checklist check
docker compose smoke
```

14. 推荐 GitHub Actions workflow

14.1 ci.yml

```
name: ci

on:
  pull_request:
  push:
    branches:
      - main

jobs:
  ci:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_USER: erp
          POSTGRES_PASSWORD: erp_test_password
          POSTGRES_DB: erp_test
        ports:
          - 5432:5432
        options: >-
          --health-cmd "pg_isready -U erp -d erp_test"
          --health-interval 10s
          --health-timeout 5s
          --health-retries 10

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 24.14.0

      - name: Enable corepack
        run: corepack enable

      - name: Prepare pnpm
        run: corepack prepare pnpm@10.13.1 --activate

      - uses: actions/setup-go@v5
        with:
          go-version: 1.26.4

      - name: Project scan
```

```

    run: bash scripts/project-scan.sh --strict

  - name: Security scan
    run: bash scripts/qa/security.sh

  - name: Script tests
    run: bash scripts/qa/scripts.sh

  - name: Backend tests
    working-directory: server
    env:
      TEST_DATABASE_URL: postgres://erp:erp_test_password@localhost:5432/erp_test?sslmode=disable
    run: bash ../scripts/qa/backend.sh

  - name: Frontend tests
    run: bash scripts/qa/frontend.sh

  - name: Docker build server
    run: docker build -f server/Dockerfile server

  - name: Docker build web
    run: docker build -f web/Dockerfile web

```

14.2 nightly.yml

```

name: nightly

on:
  schedule:
    - cron: "0 18 * * *"
  workflow_dispatch:

jobs:
  nightly:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 24.14.0

      - uses: actions/setup-go@v5
        with:
          go-version: 1.26.4

```

```
- run: corepack enable
- run: corepack prepare pnpm@10.13.1 --activate

- name: Full QA
  run: bash scripts/qa/full.sh

- name: Race tests
  working-directory: server
  run: go test -race ./...

- name: Playwright E2E
  working-directory: web
  run: pnpm e2e

- name: Backup restore drill
  run: bash scripts/deploy/verify-restore.sh

- name: Release package dry-run
  run: node scripts/release/build-customer-trial-package.mjs --dry-run
```

15. 本地 QA 命令设计

15.1 快速检查

```
bash scripts/qa/fast.sh
```

应该包含：

```
project scan
script tests
go test fast packages
web lint
web unit tests
```

15.2 完整检查

```
bash scripts/qa/full.sh
```

应该包含：

```
fast
PostgreSQL integration tests
```

```
web build
Playwright smoke
migration lint
docker build
```

15.3 严格检查

```
bash scripts/qa/strict.sh
```

应该包含：

```
full
secret scan full tree
go test -race
release package dry-run
backup restore dry-run
performance smoke
```

16. 覆盖率目标

覆盖率不要一开始追求虚高，但核心领域必须硬性要求。

16.1 初始目标

模块	覆盖率目标
config/preflight	90%+
inventory usecase	85%+
shipment usecase	85%+
purchase usecase	80%+
sales usecase	80%+
bom usecase	80%+
auth/rbac	85%+
import scripts	80%+
frontend components	60%+
whole project	不强制

16.2 不建议只看总覆盖率

不推荐这种目标：

项目整体覆盖率 80%

原因：

容易为了覆盖率测试无关 getter、DTO、简单组件，却漏掉库存并发、出货幂等等真正风险。

推荐：

核心领域必须覆盖
风险动作必须覆盖
状态机必须覆盖
事务和并发必须覆盖

17. 测试数据 Builder 设计

17.1 Go Builder 示例

```
type FixtureFactory struct {
    DB *ent.Client
}

func (f *FixtureFactory) Customer(ctx context.Context,
    opts ...CustomerOption) *ent.Customer {
    // create customer with sane defaults
}

func (f *FixtureFactory) SKU(ctx context.Context, opts ...SKUOption)
*ent.ProductSKU {
    // create product + sku if needed
}

func (f *FixtureFactory) InventoryBalance(ctx context.Context, skuID int,
    qty decimal.Decimal) *ent.InventoryBalance {
    // create balance
}
```

17.2 Builder 必须支持默认值

调用方应该能这样写：

```
customer := fx.Customer(ctx)
sku := fx.SKU(ctx)
warehouse := fx.Warehouse(ctx)
fx.InventoryBalance(ctx, sku.ID, decimal.NewFromInt(100))
```

而不是每个测试写几十行创建逻辑。

17.3 Builder 必须支持覆盖

```
customer := fx.Customer(ctx, WithCustomerStatus("INACTIVE"))
sku := fx.SKU(ctx, WithSKUCode("PLUSH-BEAR-PINK-S"))
po := fx.PurchaseOrder(ctx, WithPOStatus("APPROVED"))
```

18. Mock 策略

18.1 可以 mock 的东西

```
短信服务
邮件服务
文件存储
时间
当前用户
外部 webhook
浏览器 localStorage
前端 JSON-RPC client
```

18.2 不建议 mock 的东西

核心 integration 测试中不建议 mock：

```
PostgreSQL
库存 repository
出货 repository
采购收货 repository
事务
```

唯一约束
行锁

原因：

ERP 最容易出问题的是数据一致性、事务、并发和约束，mock 掉数据库会错过关键问题。

19. 测试中的状态机断言

每个有状态的模块必须测试：

允许的状态迁移
禁止的状态迁移
重复动作
取消动作
关闭动作
异常动作
只读状态

示例：销售订单

```
DRAFT -> CONFIRMED
CONFIRMED -> IN_PRODUCTION
CONFIRMED -> PARTIALLY_SHIPPED
PARTIALLY_SHIPPED -> SHIPPED
SHIPPED -> CLOSED
DRAFT -> CANCELLED
CONFIRMED -> CANCELLED
```

禁止：

```
SHIPPED -> DRAFT
CLOSED -> CONFIRMED
CANCELLED -> SHIPPED
```

20. 幂等测试规范

20.1 必须支持幂等的动作

```
create reservation  
cancel reservation  
add shipment item  
ship shipment  
cancel shipped shipment  
create finance fact from source  
apply import  
confirm payment  
bootstrap admin
```

20.2 幂等 key 规则

建议：

```
idempotency_key = actor + action + source_type + source_id + source_line_id
```

或由客户端传入：

```
Idempotency-Key header  
params.idempotencyKey
```

20.3 幂等测试必须断言

```
重复请求返回同一业务结果  
不会新增重复行  
不会重复扣库存  
不会重复生成财务事实  
不会重复写关键 audit event, 或 audit event 标记 retry
```

21. 并发测试规范

21.1 必须做并发测试的动作

库存预留
出货扣库存
采购收货更新 received_qty
生产上报更新 completed_qty
收付款更新 paid_amount
导入 apply

21.2 并发测试要求

必须使用 PostgreSQL
必须多次运行 -count=20 或更多
必须断言最终事实一致
必须断言失败请求不会产生副作用

22. 测试报告格式

22.1 JSON Report

建议所有脚本类测试输出 JSON report：

```
{
  "name": "manufacturing-mvp",
  "status": "passed",
  "startedAt": "2026-06-11T00:00:00Z",
  "finishedAt": "2026-06-11T00:01:00Z",
  "checks": [
    {
      "name": "salesOrderConfirmed",
      "status": "passed"
    }
  ],
  "errors": []
}
```

22.2 Release Report

release 包中应包含：

```
test-report.json
security-scan-report.json
migration-status.txt
import-dry-run-report.json
smoke-test-report.json
backup-restore-report.json
```

23. 自动化测试落地路线图

23.1 第一阶段：P0 测试底座

先做：

```
P0-T01 Secret Scan
P0-T02 Production Preflight
P0-T03 SQL Tracing Redaction
P0-T04 Import Manifest
P0-T05 CI Baseline
```

完成定义：

```
CI 可运行
secret 可拦截
危险配置可拦截
导入测试可复现
SQL 参数不进 trace
```

23.2 第二阶段：P1 核心事实保护

再做：

```
P1-T01 禁止 Runtime Phase8
P1-T02 Business Records 只读
P1-T03 Workflow 边界
P1-T04 库存预留并发
P1-T05 库存负数保护
P1-T06 出货幂等
```

P1-T07 采购收货数量
P1-T08 BOM 状态机
P1-T09 权限后端强制
P1-T10 审计日志

完成定义：

核心事实源不会被旧路径污染
库存/出货/采购/BOM/RBAC/Audit 关键风险有测试保护

23.3 第三阶段：P2 页面和主链路

继续做：

P2-T01 ERP MVP 后端 E2E
P2-T02 JSON-RPC Contract
P2-T03 前端核心页面组件
P2-T04 Playwright 核心 E2E
P2-T05 前端权限菜单

完成定义：

后端主链路可跑通
API contract 稳定
前端核心页面不回归
浏览器 smoke 可跑通

23.4 第四阶段：P3 交付测试

最后做：

P3-T01 Release Package 测试
P3-T02 备份恢复测试
P3-T03 性能 Smoke

完成定义：

客户试用包可验证
私有化部署可恢复
基础性能无明显退化

24. 最短可执行版本

如果只想先做最关键的 12 个自动化测试任务，建议按这个顺序：

1. P0-T01 Secret Scan
2. P0-T02 Production Preflight
3. P0-T03 SQL Tracing Redaction
4. P0-T04 Import Manifest
5. P0-T05 CI Baseline
6. P1-T01 禁止 Runtime Phase8
7. P1-T02 Business Records 只读
8. P1-T03 Workflow 边界
9. P1-T04 库存预留并发
10. P1-T06 出货幂等
11. P1-T09 权限后端强制
12. P2-T01 ERP MVP 后端 E2E

这 12 个完成后，项目质量会有明显跃迁。

25. Codex 通用测试任务提示词

后续每次让 Codex 补测试，可以使用这个模板：

你现在在 plush-toy-erp 项目中工作。

请只完成自动化测试方案中的任务：<任务编号和名称>。

重要约束：

1. 不允许提交任何真实 token、密码、私钥、webhook、客户凭证。
2. 不允许通过 business_records 写正式业务事实。
3. 不允许新增 phase8 runtime API、phase8 前端路由或 phase8 测试依赖。
4. Workflow 不允许直接写库存、出货、采购、财务事实。
5. 核心库存、出货、采购、BOM、权限测试必须断言数据库最终事实。
6. 并发/事务测试必须使用 PostgreSQL，不要用 SQLite 代替。
7. 如果测试无法运行，必须说明原因。
8. 完成后输出修改文件、测试命令、测试结果、失败项和风险。

任务详情：

<粘贴对应测试任务>

请按以下步骤执行：

1. 阅读现有测试和相关代码。
2. 说明测试设计。
3. 添加或修改测试。

4. 如发现代码 bug，可以最小范围修复。
5. 运行相关测试。
6. 输出结果。

26. Codex 完成后的输出格式

Codex 每次完成测试任务后，必须输出：

测试任务

- 任务编号：
- 任务名称：

测试覆盖

- 覆盖了哪些场景：
- 没覆盖哪些场景：
- 原因：

修改文件

- path/to/file1
- path/to/file2

运行命令

- command 1
- command 2

结果

- 通过：
- 失败：
- 跳过：
- 未运行：

发现的问题

- bug 1：
- bug 2：

后续建议

- 建议 1：
- 建议 2：

27. 人工 Review 清单

每个测试 PR 人工 Review 时检查：

- ☐ 测试是否真的断言了业务事实，而不是只断言无报错
- ☐ 测试是否依赖真实 secret
- ☐ 测试是否依赖本机路径
- ☐ 测试是否依赖当前时间
- ☐ 测试是否依赖运行顺序
- ☐ 测试是否会污染其他测试
- ☐ 并发测试是否使用 PostgreSQL
- ☐ 幂等测试是否断言行数和库存数量
- ☐ 权限测试是否走后端 API
- ☐ 前端权限测试是否没有替代后端权限测试
- ☐ E2E 是否太脆弱
- ☐ 测试数据是否可读
- ☐ 错误信息是否清晰
- ☐ CI 是否会运行该测试
- ☐ 如果测试暂不进 CI，是否说明原因

28. 最终目标

自动化测试最终要达到：

每次提交都能保护安全和基础质量。
每次合并都能保护核心领域事实。
每次发布都能证明客户试用包可运行、可导入、可备份、可恢复、无 secret。

一句话原则：

ERP 自动化测试的核心不是“页面能打开”，而是“业务事实不会错”。